

THE DEVELOPER PROMPT REFERENCE CARD

WRITE BETTER CODE. SOLVE FASTER.
GET BETTER AI RESULTS.



 **CODE FASTER**
Generate, refactor,
and debug code
efficiently.

 **DEBUG SMARTER**
Find issues, edge
cases, and fix
bugs faster.

 **ANY STACK**
Works with any
language, framework,
or environment.



	CLEAR GOAL Define the task and expected output.	
	CONTEXT Provide relevant background, tech stack, and constraints.	
	INPUT / OUTPUT Specify inputs, formats, and desired results.	
	CONSTRAINTS Add rules, limitations, and edge cases.	
	EXAMPLES Include examples to improve accuracy and clarity.	

 **BETTER PROMPTS**
Structured prompts
that deliver accurate
developer results.

 **BOOST PRODUCTIVITY**
Automate tasks,
save time, and ship
high-quality code.

 **RELIABLE RESULTS**
Get consistent,
production-ready
outputs every time.



Write Better
Prompts



Solve Complex
Problems



All Languages
& Frameworks



Level Up Your
Development

How to Use This Reference Card

This is your go-to AI prompting reference for the entire development workflow — from scaffolding to shipping. Every prompt is copy-paste ready. Replace the brackets, paste into ChatGPT, Claude, or GitHub Copilot Chat, and get working output immediately.

STEP 1 — Find your workflow stage

Jump to the section that matches where you are in your dev process.

STEP 2 — Copy the prompt template

Copy the prompt exactly — do not paraphrase it.

STEP 3 — Fill the [BRACKETS]

Replace every [BRACKET] with your actual language, framework, function, or context.

STEP 4 — Paste the relevant code

For debugging and review prompts — always paste the actual code.

STEP 5 — Review the output carefully

AI output is a starting point — always review before running in production.

STEP 6 — Iterate with follow-ups

Use the follow-up prompts to refine, explain, optimize, or test the output.

DEV TIP

AI is your pair programmer — not your replacement.
Always understand the code it produces before merging it.

The Developer Prompt Formula

Every great developer prompt follows the same structure. Learn this once and you can build a perfect prompt for any coding task in seconds.

LANG/STACK	TASK	CONTEXT	CONSTRAINTS	FORMAT	EXPLAIN
LANG/STACK Specify language, version, and framework. Example: Python 3.11 / React 18 + TypeScript / Node.js + Express / Go 1.21 / Rust	TASK Precise action verb + deliverable. Example: Write a function that... / Refactor this class... / Debug this error... / Review this code...	CONTEXT What does the code connect to? Example: Part of a REST API / uses PostgreSQL via SQLAlchemy / runs in a Lambda function / called by X	CONSTRAINTS What rules must the code follow? Example: No external libraries / under 50 lines / $O(n \log n)$ max / must handle null inputs / async only	FORMAT How should the output be structured? Example: Single function / class / module / with docstring / with unit tests / with type hints	EXPLAIN Do you want an explanation alongside the code? Example: After the code: explain each function / walk through the logic / note any edge cases

Code Generation Prompt Templates

From single functions to full modules — prompts that generate clean, working code the first time.

WRITE A FUNCTION

Act as a senior [LANGUAGE] developer.
Write a function called [FUNCTION_NAME] that:

- Takes [INPUT PARAMETERS with types]
- Returns [RETURN TYPE and description]
- Does [EXACT BEHAVIOUR in plain English]

Language: [LANGUAGE + VERSION]. Framework: [FRAMEWORK if applicable].
Include: type hints / annotations, docstring, input validation, error handling.
Edge cases to handle: [LIST EDGE CASES].
No external libraries beyond [LIST ALLOWED LIBRARIES].

WRITE A CLASS / MODULE

Act as a senior [LANGUAGE] developer.
Write a [CLASS / MODULE] called [NAME] for [PURPOSE].
It should: [LIST 3-5 BEHAVIOURS OR RESPONSIBILITIES].
It must implement: [INTERFACES / BASE CLASSES if applicable].
Language: [LANGUAGE]. Style: [OOP / functional / procedural].
Include: constructor, [LIST KEY METHODS], property getters/setters if needed.
Add docstrings to all public methods. Include type hints throughout.

BUILD A REST API ENDPOINT

Act as a backend developer specializing in [FRAMEWORK: FastAPI / Express / Django / Rails].
Create a [GET / POST / PUT / DELETE] endpoint at [ROUTE PATH].
It should: [DESCRIBE WHAT THE ENDPOINT DOES].
Request body / params: [DESCRIBE INPUT SCHEMA].
Response: [DESCRIBE OUTPUT SCHEMA].
Auth: [NONE / JWT / API KEY]. Database: [DATABASE + ORM].
Include: input validation, error handling (400/401/404/500), and docstring.

WRITE A DATA TRANSFORMATION

Write a [LANGUAGE] function to transform this data structure:
INPUT: [DESCRIBE OR PASTE INPUT STRUCTURE]
OUTPUT: [DESCRIBE OR PASTE DESIRED OUTPUT]
Requirements: handle missing or null fields gracefully,
do not mutate the input, include type hints.
Performance: this may run on [SMALL / LARGE] datasets — optimize accordingly.
After the function: add 3 unit tests covering normal case, empty input, and null fields.

SCAFFOLD A FULL FEATURE

Act as a senior full-stack developer.
Scaffold a complete [FEATURE NAME] feature for a [STACK] application.
Feature spec: [DESCRIBE WHAT THE FEATURE DOES].
Include all layers: [database model / migration / service layer / API endpoint / basic tests].
Stack: [FRONTEND FRAMEWORK] + [BACKEND FRAMEWORK] + [DATABASE].
Follow [REST / GraphQL / tRPC] patterns. Use [ORM NAME] for database.
Output each file separately with the filename as a comment at the top.

GENERATE BOILERPLATE / SCAFFOLDING

Generate boilerplate code for a new [PROJECT TYPE] project.
Language: [LANGUAGE]. Framework: [FRAMEWORK].
Include: project structure (list all files and folders), main entry point,
configuration file structure, example environment variables file (.env.example),
and a basic README skeleton.



Aicadmia

Style: [functional / OOP]. Linter: [ESLint / Pylint / none].

Debugging Prompt Templates

When something is broken, these prompts help you diagnose, fix, and understand the error — not just patch it.

DEBUG THIS ERROR

I am getting this error in my [LANGUAGE] [FRAMEWORK] application:
ERROR: [PASTE FULL ERROR MESSAGE AND STACK TRACE]
Here is the relevant code:
[PASTE YOUR CODE]
Context: [BRIEF DESCRIPTION OF WHAT THE CODE SHOULD DO].
1. Explain what this error means in plain English.
2. Identify the exact cause of the error.
3. Provide the fixed code with the changes highlighted.
4. Explain why the fix works.
5. What should I check to prevent this type of error in future?

FIND THE BUG

Act as an expert [LANGUAGE] debugger.
This code is supposed to [EXPECTED BEHAVIOUR] but instead it [ACTUAL BEHAVIOUR].
Find the bug(s) in this code:
[PASTE YOUR CODE]
Tell me:
1. The exact location and line number of the bug.
2. Why this code is wrong – the root cause, not just the symptom.
3. The corrected code.
4. A brief test I can add to prevent this bug regressing.

EXPLAIN AN ERROR MESSAGE

Explain this error message to me in plain English:
ERROR: [PASTE ERROR MESSAGE]
Language/Runtime: [LANGUAGE]. Framework: [FRAMEWORK].
Tell me:
1. What this error type means (in simple terms).
2. The most common causes of this specific error.
3. Ordered list of things to check – most likely cause first.
4. The one question I should ask myself first when I see this error.

PERFORMANCE DEBUGGING

Act as a performance engineer.
This [LANGUAGE] code is running too slowly. Target: [X ms / X seconds / X ops per second].
Current performance: [CURRENT TIMING].
Here is the code:
[PASTE YOUR CODE]
1. Identify the performance bottleneck(s) – explain WHY they are slow.
2. Suggest [X] specific optimizations, ordered by impact.
3. Show the optimized version of the slowest section.
4. What is the expected performance after your changes?

FIX MY FAILING TESTS

My [TESTING FRAMEWORK] tests are failing. Here are the failing tests and errors:
[PASTE FAILING TESTS AND OUTPUT]
Here is the code under test:
[PASTE PRODUCTION CODE]
1. Explain why each test is failing.
2. Is the bug in the test or in the production code?
3. Fix whichever is wrong and explain your reasoning.
4. Suggest one additional test case that would have caught this earlier.

Code Review Prompt Templates

Get a thorough code review without waiting for a teammate. These prompts catch bugs, security issues, and code quality problems before they reach production.

FULL CODE REVIEW

Act as a senior [LANGUAGE] engineer doing a thorough code review.
Review this code:
[PASTE YOUR CODE]
Language: [LANGUAGE]. Framework: [FRAMEWORK]. Context: [WHAT THIS CODE DOES].
Review and score each dimension (1-10 with comments):
1. Correctness – does it do what it should?
2. Readability – is it clear and well-named?
3. Performance – any inefficiencies?
4. Security – any vulnerabilities?
5. Error handling – edge cases covered?
6. Testability – is it easy to test?
End with: top 3 changes I must make before this ships.

SECURITY AUDIT

Act as a security engineer performing a security code review.
Audit this [LANGUAGE] code for security vulnerabilities:
[PASTE YOUR CODE]
Check for: [SQL injection / XSS / CSRF / insecure direct object references / authentication bypasses / sensitive data exposure / rate limiting issues / input validation / dependency vulnerabilities].
For each vulnerability found:
- Severity: CRITICAL / HIGH / MEDIUM / LOW
- Exact location in the code
- Explanation of the risk
- Fixed code snippet

REVIEW FOR PERFORMANCE

Act as a performance engineer.
Review this code for performance issues:
[PASTE YOUR CODE]
Language: [LANGUAGE]. Scale: this will handle [EXPECTED LOAD].
Identify: unnecessary loops, N+1 queries, blocking operations, memory leaks, unoptimized algorithms, redundant computations.
For each issue: severity rating, explanation, and the optimized alternative.

REFACTOR SUGGESTIONS

Act as a senior [LANGUAGE] developer who cares deeply about clean code.
Suggest refactoring improvements for this code:
[PASTE YOUR CODE]
Focus on: function/variable naming, SOLID principles, DRY violations, function length and single responsibility, magic numbers, and dead code.
For the top 3 most impactful refactors: show the before and after.
Do NOT change the behaviour – only improve the structure.

API CONTRACT REVIEW

Review this API design for best practices:
[PASTE YOUR API SPEC, ROUTES, OR ENDPOINT DEFINITIONS]
Check: naming conventions (RESTful / GraphQL conventions), consistent response structure, proper HTTP status codes, error response format, versioning strategy, authentication approach, and any missing standard fields.
Give a score and specific actionable improvements for each area.

Before & After: Developer Prompts

See the difference between a prompt that gets you a generic snippet and one that gets you production-ready code.

✗ Weak

Write a login function.

✓ Strong

Act as a senior Node.js security engineer. Write an async login function using Express + JWT. It should: accept email + password, query a PostgreSQL users table via pg, compare password with bcrypt, return a signed JWT (24h expiry) on success, return 401 on wrong credentials, 400 on missing fields. Add input validation (no empty strings). Include: JSDoc, error handling, and one unit test for wrong password. No external libraries beyond pg, bcrypt, jsonwebtoken.

✗ Weak

Help me fix this error: TypeError: Cannot read properties of undefined.

✓ Strong

Act as an expert JavaScript debugger. I am getting this error: "TypeError: Cannot read properties of undefined (reading 'map')" on line 47. The code that fails is: [PASTE CODE]. Context: this runs in a React component fetching data from a REST API. 1. Explain exactly why this error occurs. 2. Identify the root cause (is data undefined at render time?). 3. Provide the fixed code. 4. Add a guard that prevents this class of error in future.

✗ Weak

Review my code.

✓ Strong

Act as a senior Python engineer. Do a thorough code review of this FastAPI endpoint: [PASTE CODE]. Score each: correctness (is it doing what it should?), security (any injection risks or auth issues?), performance (any N+1 or blocking ops?), error handling (all edge cases covered?). Score each 1-10 with specific comments. List top 3 must-fix items before this goes to production.

DEV TIP

Always include: language + version + framework + context + constraints.
These 5 additions transform a generic snippet into production-ready code.

Refactoring Prompt Templates

Clean up legacy code, improve structure, reduce complexity, and make code easier to maintain — without changing behaviour.

REFACTOR FOR READABILITY

Act as a senior [LANGUAGE] developer focused on clean code.

Refactor this code for maximum readability:

[PASTE YOUR CODE]

Rules: do not change the external behaviour or function signatures.

Improve: variable and function names, extract magic numbers to named constants, break large functions into smaller ones (single responsibility), add comments only where the logic is genuinely non-obvious.

Show: the refactored version with a brief explanation of each key change.

REDUCE COMPLEXITY

Act as a code quality engineer.

This function has a cyclomatic complexity that is too high:

[PASTE YOUR FUNCTION]

Reduce the complexity by: extracting helper functions, using early returns, simplifying conditionals, and reducing nesting.

Target: cyclomatic complexity under [X].

Constraints: same input/output behaviour, same language + version.

Show before (annotated) and after (with explanation).

MODERNIZE LEGACY CODE

Act as a [LANGUAGE] modernization expert.

Update this legacy [LANGUAGE VERSION] code to modern [TARGET VERSION] standards:

[PASTE YOUR CODE]

Target version: [TARGET VERSION]. Breaking changes allowed: [YES / NO].

Update: syntax, APIs, deprecated patterns, and style.

Highlight every change with a comment explaining what changed and why.

Flag any changes that could affect behaviour.

EXTRACT REUSABLE MODULE

Act as a senior [LANGUAGE] architect.

This code contains logic that should be extracted into a reusable module:

[PASTE YOUR CODE]

Identify: what logic is generic enough to reuse, what should stay specific.

Create: a clean, well-documented module / utility file with the extracted logic.

Update: the original code to use the new module.

Add: JSDoc / docstrings, type hints, and a simple usage example.

CONVERT TO ASYNC / AWAIT

Convert this [LANGUAGE] code from [CALLBACKS / PROMISES / SYNC] to async/await:

[PASTE YOUR CODE]

Requirements: same behaviour and error handling, no new external dependencies.

Handle: all error cases with try/catch, any parallel operations with Promise.all, timeout or cancellation if needed.

Explain any behaviour changes introduced by the async conversion.

Testing Prompt Templates

Unit tests, integration tests, edge cases, and test strategies — write better tests faster with AI.

WRITE UNIT TESTS

Act as a senior [LANGUAGE] developer who writes excellent tests.
Write unit tests for this function using [TESTING FRAMEWORK]:
[PASTE YOUR FUNCTION]
Test coverage requirements:
- Happy path (normal expected usage)
- Empty / null / undefined inputs
- Boundary values (min, max, zero)
- Error / exception cases
- Any asynchronous behaviour
Use: [describe / it / test] blocks, mocking for [LIST DEPENDENCIES].
Aim for 100% branch coverage. Do not test implementation details.

WRITE INTEGRATION TESTS

Act as a [FRAMEWORK] integration test specialist.
Write integration tests for this [API ENDPOINT / FEATURE / MODULE]:
[PASTE YOUR CODE OR DESCRIBE THE FEATURE]
Test: the complete flow from [ENTRY POINT] to [EXPECTED OUTCOME].
Include: database setup/teardown, HTTP request simulation, auth token handling.
Cover: success case, missing auth, invalid input, resource not found, server error.
Framework: [TEST FRAMEWORK]. Database: [DATABASE – use in-memory / test DB].

GENERATE EDGE CASES

Analyze this function and identify all possible edge cases I should test:
[PASTE YOUR FUNCTION]
For each edge case:
1. Describe the scenario
2. Why it might cause a failure
3. Write the test code for it
Think about: extreme values, type coercion, race conditions, empty collections,
very large inputs, special characters, concurrent calls.

WRITE MOCK / STUB

Create mocks and stubs for the following dependencies in [TESTING FRAMEWORK]:
Dependencies to mock: [LIST WHAT NEEDS MOCKING – e.g. database, external API, file system].
They are used in this function:
[PASTE YOUR FUNCTION]
Requirements: mocks should be realistic (plausible data), reusable across tests,
easy to reset between test cases.
Show: the mock setup code + one example test using the mocks.

TEST COVERAGE ANALYSIS

Analyze this code for test coverage gaps:
[PASTE YOUR CODE + ANY EXISTING TESTS]
Tell me:
1. Which branches are currently untested?
2. Which error paths are missing test coverage?
3. What are the 3 highest-risk untested scenarios?
4. Write the 3 missing tests that would have the highest impact on confidence.

Documentation Prompt Templates

From inline comments to full READMEs to API docs — write documentation that developers actually read.

WRITE DOCSTRINGS / JSDOC

Write [docstrings / JSDoc / Javadoc / XML doc comments] for all public functions in this [LANGUAGE] code:
[PASTE YOUR CODE]
For each function include: description, all @param tags with types and descriptions, @returns tag with type and description, @throws for any exceptions, @example with a real usage example. Follow [Google style / NumPy style / Sphinx style] conventions.

WRITE A README

Write a professional README.md for this project:
Project name: [NAME]. What it does: [BRIEF DESCRIPTION].
Tech stack: [LIST STACK]. Main features: [LIST 3-5 FEATURES].
Include: badges placeholder, description, prerequisites, installation (step-by-step), usage with code examples, API reference (basic), configuration options, contributing guide, and license section.
Tone: clear and welcoming to contributors.

WRITE API DOCUMENTATION

Write API documentation for these endpoints:
[PASTE YOUR ROUTES / API SPEC]
Format: OpenAPI / Swagger compatible markdown or YAML.
For each endpoint: method + path, description, request parameters (query/body/headers), response schema (200 + error codes), authentication required, and one curl example.

EXPLAIN COMPLEX CODE

Act as a patient senior developer explaining code to a junior developer.
Explain what this code does, step by step:
[PASTE YOUR CODE]
Format: line-by-line or block-by-block explanation.
Include: what each function does, why specific patterns were chosen, any non-obvious algorithmic decisions, and what I should understand before modifying this code. Use analogies where helpful.

WRITE INLINE COMMENTS

Add clear, helpful inline comments to this code:
[PASTE YOUR CODE]
Rules: comment the WHY, not the WHAT (assume the reader knows [LANGUAGE]).
Only comment non-obvious logic, complex algorithms, or important constraints.
Do not comment self-explanatory code like variable assignments.
Language: [LANGUAGE]. Team level: [JUNIOR / MID / SENIOR] readers.

Architecture & System Design Prompts

Design systems, choose patterns, review architecture decisions, and plan technical roadmaps with AI as your architecture partner.

SYSTEM DESIGN REVIEW

Act as a senior software architect.

Review this system design:

[DESCRIBE YOUR SYSTEM OR PASTE YOUR DESIGN DOCUMENT]

Scale requirements: [USERS / REQUESTS PER SECOND / DATA VOLUME].

Evaluate: scalability bottlenecks, single points of failure, data consistency approach, caching strategy, security model, observability gaps.

For each concern: severity (P1/P2/P3), explanation, and recommended solution.

CHOOSE A DESIGN PATTERN

I need to implement [DESCRIBE THE PROBLEM] in [LANGUAGE/Framework].

Recommend the most appropriate design pattern(s).

For your top recommendation:

- Why this pattern fits this problem
- Simple implementation example in [LANGUAGE]
- When NOT to use this pattern
- Alternative patterns and why you ruled them out

Context: [ADDITIONAL CONTEXT ABOUT CODEBASE / TEAM / CONSTRAINTS].

DATABASE SCHEMA DESIGN

Act as a database architect.

Design a [RELATIONAL / DOCUMENT / GRAPH] database schema for [DESCRIBE THE SYSTEM].

Requirements: [LIST CORE ENTITIES AND RELATIONSHIPS].

Scale: [EXPECTED ROWS / USERS / QUERIES PER SECOND].

Include: table/collection definitions, relationships (FK constraints / refs), indexes (justify each one), any denormalization decisions, and migration notes.

Database: [PostgreSQL / MySQL / MongoDB / etc.].

API DESIGN

Act as an API design expert.

Design a [REST / GraphQL / gRPC] API for [DESCRIBE THE FEATURE / SYSTEM].

Requirements: [LIST WHAT THE API NEEDS TO DO].

Include: endpoint definitions, request/response schemas, error format, authentication approach, versioning strategy, and rate limiting plan.

Follow [REST best practices / GraphQL conventions].

Highlight any design decisions that need team discussion.

MICROSERVICES VS MONOLITH

Act as a solutions architect.

Evaluate whether [SYSTEM DESCRIPTION] should use microservices or remain a monolith.

Team size: [X]. Current scale: [SCALE]. Expected growth: [GROWTH].

Evaluate both options honestly:

- Monolith: pros in this context / cons / when to reconsider
- Microservices: pros in this context / cons / operational overhead

Give a clear recommendation with your reasoning.

SQL & Database Prompt Templates

Write queries, optimize slow SQL, design schemas, and manage migrations — with AI as your database expert.

WRITE A SQL QUERY

Write a [DATABASE: PostgreSQL / MySQL / SQLite] query that:
[DESCRIBE WHAT THE QUERY SHOULD DO]
Tables and their relevant columns:
[DESCRIBE YOUR SCHEMA OR PASTE TABLE DEFINITIONS]
Requirements: [FILTERING CONDITIONS], [SORTING], [PAGINATION if needed].
Performance: this query may run against [ROWS COUNT] rows.
Return: the query + a brief explanation of any non-obvious joins or subqueries.

OPTIMIZE A SLOW QUERY

Act as a database performance engineer.
This SQL query is running too slowly (current runtime: [X seconds]):
[PASTE YOUR QUERY]
Database: [DATABASE]. Approximate table sizes: [ROW COUNTS].
Existing indexes: [LIST INDEXES if known].
1. Explain WHY the query is slow (query plan analysis).
2. Suggest specific index additions with the CREATE INDEX statements.
3. Rewrite the query to be more efficient if possible.
4. What EXPLAIN ANALYZE output should I look for to verify improvement?

WRITE A DATABASE MIGRATION

Write a database migration for [ORM: Alembic / Knex / Flyway / ActiveRecord].
Change required: [DESCRIBE THE SCHEMA CHANGE].
Current schema: [PASTE RELEVANT CURRENT SCHEMA].
Include: up migration (forward), down migration (rollback),
data migration if needed, and any index changes.
Be safe: mark any destructive operations clearly. Default null values for new NOT NULL columns.

DESIGN INDEXING STRATEGY

Act as a database architect.
Recommend an indexing strategy for this query workload:
[DESCRIBE YOUR MOST COMMON QUERIES – what columns appear in WHERE, JOIN, ORDER BY, GROUP BY].
Table: [TABLE NAME]. Row count: [APPROX ROWS]. Read/write ratio: [X:Y].
For each recommended index:
- The exact CREATE INDEX statement
- Which query(ies) it helps
- Estimated write overhead
- Whether it should be partial or composite

EXPLAIN QUERY PLAN

Act as a PostgreSQL / MySQL performance expert.
Explain this query execution plan in plain English:
[PASTE YOUR EXPLAIN / EXPLAIN ANALYZE OUTPUT]
Tell me:
1. What the database is actually doing to execute this query
2. Which step is the most expensive and why
3. What the Seq Scan / Index Scan / Hash Join / etc. terms mean here
4. Specific changes that would make this plan faster

Language-Specific Prompt Packs

Pre-tuned prompts for the most common languages — with the right terminology, conventions, and context baked in.

PYTHON

- "Write a Pythonic solution for [TASK]. Use list comprehensions / generators where appropriate. Add type hints and a docstring. Python 3.11."
- "Optimize this Python code for performance. Consider: vectorization with NumPy, replacing loops with built-ins, or using [functools / itertools]."
- "Debug this Python error: [ERROR]. Explain the traceback and fix the root cause."
- "Write a pytest test for this function. Cover: happy path, edge cases, and mock [DEPENDENCY]."
- "Refactor this Python class to follow SOLID principles. Show before and after."

JAVASCRIPT / TYPESCRIPT

- "Write a TypeScript function with full type safety for [TASK]. Use generics if appropriate. No any types."
- "Convert this JavaScript to TypeScript. Add types, interfaces, and fix any implicit any issues."
- "Debug this async/await code — it is not handling errors correctly: [PASTE CODE]"
- "Write a Jest unit test for this [React component / utility function]. Mock [DEPENDENCIES]."
- "Optimize this React component to prevent unnecessary re-renders. Use memo / useMemo / useCallback where needed."

PYTHON (DJANGO / FASTAPI)

- "Write a Django REST Framework serializer for [MODEL] with [FIELD] validation."
- "Create a FastAPI endpoint with Pydantic request/response models, dependency injection, and proper HTTP status codes for [ENDPOINT]."
- "Write a Django migration for [SCHEMA CHANGE] that handles existing data safely."
- "Add rate limiting to this FastAPI endpoint using slowapi. Max [X] requests per [TIME]."
- "Write Django ORM queries to [DESCRIBE QUERY] — avoid N+1 queries. Use select_related / prefetch_related."

GO

- "Write idiomatic Go code for [TASK]. Use goroutines / channels where appropriate. Follow Go conventions."
- "Handle errors properly in this Go function. Use custom error types and wrap errors with context."
- "Write a Go interface for [DESCRIBE WHAT IT SHOULD DO] and two implementations."
- "Optimize this Go code for concurrency. Identify race conditions using comments before fixing."
- "Write a Go benchmark test for [FUNCTION] using testing.B."

RUST

- "Write safe Rust code for [TASK]. Avoid unsafe blocks. Handle ownership and borrowing correctly."
- "Fix these Rust compiler errors: [PASTE ERRORS + CODE]. Explain why each error occurred."
- "Implement [TRAIT] for [STRUCT] in Rust with full error handling."
- "Rewrite this Rust function to use async/await with tokio. Handle all Result types explicitly."
- "Explain this Rust lifetime annotation: [PASTE FUNCTION SIGNATURE]. Why is it needed?"

Before & After: Advanced Dev Prompts

The difference between a prompt that gets toy code and one that gets production-ready, maintainable software.

✗ Weak Write an auth system.	✓ Strong Act as a senior Node.js security engineer. Implement JWT authentication middleware for an Express API. Requirements: verify JWT in Authorization Bearer header, reject expired tokens (401) and invalid signatures (401), attach decoded user object to req.user, handle missing header (401). Use jsonwebtoken library. Include TypeScript types, JSDoc, and a unit test for each rejection case. Do NOT store tokens server-side – stateless only.
✗ Weak Help me with database stuff.	✓ Strong Act as a PostgreSQL performance engineer. This query takes 8 seconds on a 2M row table: [PASTE QUERY]. Current indexes: [LIST INDEXES]. 1. Explain why the query is slow using query plan analysis. 2. Write the CREATE INDEX statements that would help. 3. Rewrite the query to be more efficient. 4. What EXPLAIN ANALYZE metrics should improve and by how much?
✗ Weak Review this code.	✓ Strong Act as a senior Python engineer reviewing a pull request. Review this FastAPI endpoint: [PASTE CODE]. Score each dimension 1-10 with specific comments: correctness, security (check for injection, missing auth), performance (N+1 queries, blocking ops), error handling (missing status codes, unhandled exceptions), and code style (PEP 8, naming, complexity). List exactly 3 changes required before merge.

DevOps & CI/CD Prompt Templates

Docker, CI/CD pipelines, infrastructure as code, and deployment configurations — built and reviewed with AI.

WRITE A DOCKERFILE

Write an optimized Dockerfile for a [LANGUAGE / FRAMEWORK] application.
App type: [WEB SERVER / WORKER / CLI / etc.].
Requirements: [LIST DEPENDENCIES / SYSTEM PACKAGES NEEDED].
Optimization goals: minimize image size, maximize layer caching, security best practices.
Include: multi-stage build if beneficial, non-root user, .dockerignore suggestions, HEALTHCHECK instruction, and ENV variable placeholders.
Base image recommendation with reasoning.

WRITE A GITHUB ACTIONS WORKFLOW

Write a GitHub Actions CI/CD workflow for a [LANGUAGE / FRAMEWORK] project.
Triggers: [PUSH TO MAIN / PR TO MAIN / MANUAL].
Pipeline stages: [lint / test / build / deploy].
Requirements: [LIST SPECIFIC REQUIREMENTS — e.g. Docker push, deploy to AWS, notify Slack].
Secrets used: [LIST SECRET NAMES].
Include: caching for [package manager], parallel jobs where possible, fail fast on errors.

WRITE KUBERNETES MANIFEST

Write Kubernetes manifests for deploying [APPLICATION NAME].
App type: [DEPLOYMENT / STATEFULSET / DAEMONSET].
Replicas: [X]. Image: [IMAGE NAME:TAG].
Requirements: [LIST REQUIREMENTS — e.g. persistent storage, env from secrets, readiness probe].
Include: Deployment / Service / Ingress / ConfigMap / Secret templates.
Resources: CPU limit [X]m, memory limit [X]Mi.
Add: liveness and readiness probes.

WRITE TERRAFORM RESOURCE

Write Terraform code to provision [RESOURCE TYPE] on [AWS / GCP / AZURE].
Resource spec: [DESCRIBE WHAT YOU NEED — e.g. RDS instance, S3 bucket, ECS service].
Variables to parameterize: [LIST WHAT SHOULD BE VARIABLE].
Include: resource definition, variable declarations, outputs, and locals.
Follow Terraform best practices: no hardcoded values, appropriate tags, state backend note.

WRITE DOCKER COMPOSE

Write a docker-compose.yml for a [DESCRIBE YOUR STACK — e.g. Node.js + PostgreSQL + Redis] development environment.
Services: [LIST SERVICES].
Requirements: [VOLUME MOUNTS / PORT MAPPINGS / ENV VARIABLES / DEPENDS_ON / HEALTHCHECKS].
Include: a .env.example file structure comment, network definition, named volumes for persistence, and override file suggestion for production.

Regex, Data & Algorithm Prompts

Write regex patterns, transform data structures, implement algorithms, and solve computational problems with AI.

WRITE A REGEX PATTERN

Write a [LANGUAGE-FLAVORED] regex pattern that matches: [DESCRIBE WHAT TO MATCH].
Must match: [LIST VALID EXAMPLES].
Must NOT match: [LIST INVALID EXAMPLES].
Flags needed: [g / i / m / s - explain if unsure].
Provide: the pattern, a [LANGUAGE] code snippet using it, and an explanation of each part of the regex using inline comments.

IMPLEMENT AN ALGORITHM

Act as a computer science professor and senior [LANGUAGE] developer.
Implement [ALGORITHM NAME] in [LANGUAGE].
Input: [DESCRIBE INPUT]. Output: [DESCRIBE OUTPUT].
Requirements: [TIME COMPLEXITY TARGET]. Space complexity: [TARGET or "minimize"].
Include: implementation, time and space complexity analysis (Big O),
3 test cases (base case, normal case, edge case), and explanation of the approach.

TRANSFORM DATA STRUCTURE

Write a [LANGUAGE] function to transform this data:
INPUT SHAPE:
[PASTE INPUT EXAMPLE AS JSON / CODE]
OUTPUT SHAPE:
[PASTE DESIRED OUTPUT EXAMPLE]
Handle: missing fields (use null or default), nested arrays, type coercion.
Do NOT mutate the input. Add type definitions. Under [X] lines.

DATA VALIDATION SCHEMA

Write a validation schema using [ZOD / JOI / PYDANTIC / YUP / JSONSCHEMA] for this data structure:
[PASTE YOUR DATA STRUCTURE OR DESCRIBE IT]
Rules: [LIST VALIDATION RULES - required fields, types, min/max, patterns, enum values].
Include: custom error messages for each validation rule, a validation function,
and [3] test cases covering valid data, missing required fields, and type mismatch.

SORT & SEARCH OPTIMIZATION

I have [X] records of [DATA TYPE] that I need to [SEARCH / SORT / FILTER] efficiently.
Current approach: [DESCRIBE CURRENT METHOD].
Constraints: [IN-MEMORY / CAN USE INDEX / REAL-TIME / BATCH].
Recommend: the optimal data structure and algorithm for this use case,
implement it in [LANGUAGE], and explain the performance characteristics.
Expected query pattern: [HOW IT WILL BE QUERIED].

Security Prompt Templates

Write secure code from the start. These prompts help you implement security correctly, audit for vulnerabilities, and understand security patterns.

IMPLEMENT AUTHENTICATION

Act as a security-focused senior [LANGUAGE] developer.
Implement [JWT / session-based / OAuth 2.0 / API key] authentication for [FRAMEWORK].
Requirements: [LIST SPECIFIC REQUIREMENTS – HTTPS only, token refresh, CORS, etc.].
Security requirements: secure token storage, short expiry + refresh tokens, brute force protection, secure password hashing (bcrypt / argon2).
Include: middleware, token generation, refresh endpoint, logout (token invalidation).
Flag any trade-offs between security and convenience.

IMPLEMENT AUTHORIZATION

Act as a security engineer.
Implement [RBAC / ABAC / permission-based] authorization for [SYSTEM].
Roles/permissions: [DESCRIBE YOUR ROLES AND WHAT EACH CAN DO].
Framework: [FRAMEWORK]. Language: [LANGUAGE].
Include: permission definitions, middleware / decorator / guard implementation, database schema for roles/permissions, and test cases for each role boundary.

SANITIZE USER INPUT

Act as a security engineer focused on injection prevention.
Write input sanitization and validation for [CONTEXT: web form / API endpoint / file upload].
Language: [LANGUAGE]. Framework: [FRAMEWORK].
Threats to defend against: [SQL injection / XSS / command injection / path traversal / etc.].
Input types: [LIST YOUR INPUT FIELDS AND TYPES].
Show: before (vulnerable) and after (secure) with explanation of each fix.

ENCRYPT SENSITIVE DATA

Act as a cryptography and security engineer.
Implement [ENCRYPTION AT REST / IN TRANSIT / END-TO-END] for [WHAT DATA].
Language: [LANGUAGE]. Requirements: [SPECIFIC REQUIREMENTS].
Use: [AES-256 / RSA / ECDH / etc.] – recommend if unsure.
Include: key management approach (where to store keys, rotation), encryption and decryption functions, and any performance notes.
Flag: any cryptographic choices I should get reviewed by a dedicated security team.

RATE LIMITING IMPLEMENTATION

Implement rate limiting for [ENDPOINT / API / FEATURE] using [LANGUAGE + FRAMEWORK].
Limits: [X] requests per [TIME WINDOW] per [IP / USER / API KEY].
Strategy: [fixed window / sliding window / token bucket].
Storage: [Redis / in-memory / database].
Include: middleware implementation, response headers (X-RateLimit-*), 429 Too Many Requests response format, and exemption list support.

Frontend Development Prompt Templates

React, TypeScript, CSS, accessibility, and performance — frontend prompts that produce production-quality UI code.

REACT COMPONENT

Act as a senior React developer.
Create a [COMPONENT NAME] component.
Props: [LIST PROPS WITH TYPES AND DESCRIPTIONS].
Behaviour: [DESCRIBE WHAT IT DOES].
Requirements: TypeScript, functional component with hooks, accessible (WCAG 2.1 AA), no inline styles (use [CSS modules / Tailwind / styled-components]).
Include: proper PropTypes / interface definition, loading and error states, event handlers with correct types, and a usage example.

STATE MANAGEMENT

Act as a senior React developer.
Implement state management for [FEATURE DESCRIPTION].
Current pain: [DESCRIBE THE STATE PROBLEM].
Stack: React [VERSION] + [Redux Toolkit / Zustand / Jotai / Context API / etc.].
Requirements: [LIST SPECIFIC REQUIREMENTS – performance, persistence, SSR compatibility].
Show: store/slice definition, selectors, actions, and a component using the state.
Note any performance implications of the chosen approach.

CSS ANIMATION / TRANSITION

Write CSS / [Tailwind / styled-components] for this animation:
Effect: [DESCRIBE THE ANIMATION – slide in from left, fade, scale, etc.].
Trigger: [HOVER / FOCUS / CLASS TOGGLE / SCROLL INTO VIEW].
Duration: [X ms]. Easing: [ease / ease-in-out / spring / custom].
Requirements: respect prefers-reduced-motion, hardware-accelerated (use transform/opacity), no layout thrashing. Show the CSS + a React hook / intersection observer if needed.

ACCESSIBILITY AUDIT

Act as an accessibility (a11y) expert.
Audit this [COMPONENT / PAGE] for WCAG 2.1 AA compliance:
[PASTE YOUR JSX / HTML CODE]
Check: keyboard navigation, ARIA roles and labels, color contrast, focus management, screen reader announcements, error messages, form labels.
For each issue: severity (critical / major / minor), explanation, and fixed code.

PERFORMANCE OPTIMIZATION — REACT

Act as a React performance engineer.
This component has performance issues:
[PASTE YOUR COMPONENT]
Symptoms: [RE-RENDERS TOO OFTEN / SLOW INITIAL LOAD / LARGE BUNDLE / etc.].
Identify and fix: unnecessary re-renders (add React.memo / useMemo / useCallback), expensive computations, large dependencies, missing lazy loading.
Show: before and after, and explain what each optimization does.

Git & Version Control Prompt Templates

Write better commit messages, PR descriptions, changelogs, and Git workflows with AI.

WRITE A COMMIT MESSAGE

Write a Git commit message for these changes:
[DESCRIBE OR PASTE WHAT CHANGED – can paste git diff]
Format: [Conventional Commits / Gitmoji / free-form].
Rules: imperative mood, under 72 characters for subject,
explain the WHY in the body (not just what), reference issue [#NUMBER] if applicable.
Give me 3 options from: minimal (just the subject) to detailed (with body).

WRITE A PULL REQUEST DESCRIPTION

Write a pull request description for this change:
Branch: [BRANCH NAME]. Base: [main / develop].
Change summary: [DESCRIBE WHAT YOU CHANGED AND WHY].
Related issues: [LIST ISSUE NUMBERS].
Include: what changed and why (not just what), how to test it manually,
screenshots / demo notes if UI changed, any breaking changes or migration notes,
and a checklist (tests pass, docs updated, reviewed by).

WRITE A CHANGELOG ENTRY

Write a CHANGELOG.md entry for version [X.X.X].
Format: [Keep a Changelog / Semantic Versioning].
Changes this release:
- New features: [LIST]
- Bug fixes: [LIST]
- Breaking changes: [LIST]
- Deprecated: [LIST]
Write each item as: one-line summary with PR/issue reference where relevant.
Include a migration note for any breaking change.

GIT WORKFLOW DESIGN

Act as a senior DevOps engineer.
Design a Git branching strategy for [TEAM SIZE] developers working on [PROJECT TYPE].
Requirements: [RELEASE CADENCE / CI-CD SETUP / ENVIRONMENTS – dev/staging/prod].
Recommend: branch naming conventions, merge strategy (merge / rebase / squash),
when to cut releases, hotfix process, and code review rules.
Compare: [GitFlow / GitHub Flow / trunk-based] and recommend the best fit.

RESOLVE A MERGE CONFLICT

Help me resolve this Git merge conflict:
[PASTE THE CONFLICTED FILE WITH <<<< ==== >>>> MARKERS]
Context: [WHAT EACH BRANCH WAS TRYING TO DO].
1. Explain what each version is trying to do.
2. Show the correct resolved version.
3. Explain your resolution logic.
4. What should I test after resolving to make sure both changes are preserved?

Code Explanation & Learning Prompts

Understand unfamiliar code, learn new concepts, and deepen your technical knowledge with AI as your patient mentor.

EXPLAIN THIS CODE

Act as a patient senior developer.
Explain what this code does, step by step:
[PASTE YOUR CODE]
Assume I understand [LANGUAGE] basics but am unfamiliar with [SPECIFIC PATTERN / LIBRARY].
Explain: what each section does, why specific choices were made,
any non-obvious behaviour or gotchas, and what I should understand
before I modify or extend this code.

EXPLAIN A CONCEPT

Explain [TECHNICAL CONCEPT] to a developer who understands [LANGUAGE] basics.
Use: a plain English explanation, a real-world analogy, and a code example in [LANGUAGE].
Then: show a common mistake developers make with this concept
and how to identify and fix it. Keep the explanation under 300 words.

COMPARE TWO APPROACHES

Compare these two approaches for [PROBLEM]:
Approach A: [DESCRIBE OR PASTE CODE]
Approach B: [DESCRIBE OR PASTE CODE]
Compare: performance (Big O), readability, maintainability, memory usage, and when each
shines.
Give a clear recommendation for [MY SPECIFIC USE CASE: describe it].
Show a table summarizing the comparison.

TEACH ME THIS LIBRARY

Act as an expert in [LIBRARY / FRAMEWORK NAME].
Give me a practical introduction assuming I know [LANGUAGE] but am new to [LIBRARY].
Cover: what problem it solves, core concepts (maximum 5), most important APIs with examples,
most common beginner mistakes, and the one thing most tutorials get wrong.
Finish with a 20-line example that demonstrates the most important features.

CODE KATA / PRACTICE PROBLEM

Give me a coding challenge to practice [SKILL / CONCEPT] in [LANGUAGE].
My current level: [BEGINNER / INTERMEDIATE / ADVANCED].
Difficulty: [EASY / MEDIUM / HARD].
Include: problem description, example input and output, constraints,
a hint (hidden in a comment), and the optimal solution with explanation.
Make the problem realistic – something I might encounter at work.

Best AI Tools for Developers 2026

Not all AI tools are equal for development work. Here is the honest breakdown of what each tool does best.

Tool	Best For	Free?	Verdict
GitHub Copilot	In-IDE completion	\$10/mo	Best for daily coding
Cursor	AI-first code editor	Free + Pro	Best overall IDE
Claude Pro	Code review + explain	\$20/mo	Best for understanding code
ChatGPT Plus	General coding + debug	\$20/mo	Best for varied tasks
Gemini (Google)	Google Cloud + Android	Free	Best free tier
Codeium	Free Copilot alternative	Free	Best for budget dev
Tabnine	Local / private code	Free + Pro	Best for privacy
Replit AI	Browser-based projects	Free + Pro	Best for prototyping

GITHUB COPILOT vs CURSOR

Copilot: plug-in to your existing IDE. Great autocompletion, in-file chat.

Cursor: built-from-scratch AI IDE. Can edit multiple files, understands the whole codebase.

Verdict: Cursor for serious AI-native development. Copilot for augmenting your existing workflow.

CLAUDE for CODE REVIEW

Claude has the largest context window (200K tokens) — ideal for:

- Pasting entire files or modules for review
- Understanding complex codebases section by section
- Getting nuanced, careful architectural feedback

Best prompt: "Act as a senior engineer doing a thorough PR review of this code: [PASTE]"

CHATGPT for DEBUGGING

ChatGPT is best at: interpreting error messages, suggesting fixes, explaining stack traces.

Tip: always paste the FULL error + the relevant code section + what you expected to happen.

ChatGPT with Code Interpreter can also run Python directly to test solutions.

Developer Follow-Up Prompt Library

The first AI response is a draft. Use these follow-up prompts to improve, harden, and extend any code output.

Improve the code:

F01 Add error handling for [SPECIFIC EDGE CASE] to this code.

F02 Add input validation — reject [NULL / EMPTY / OUT OF RANGE / WRONG TYPE] inputs.

F03 Rewrite this to be more idiomatic [LANGUAGE]. Follow [LANGUAGE] conventions strictly.

F04 This function is too long. Break it into smaller, single-responsibility functions.

F05 Add logging at [DEBUG / INFO / ERROR] level at the key decision points.

Make it production-ready:

P01 Add retry logic with exponential backoff for the network calls in this code.

P02 Add structured logging in JSON format — include request ID, user ID, and timestamp.

P03 Make this code thread-safe / safe for concurrent access.

P04 Add a circuit breaker pattern for calls to [EXTERNAL SERVICE].

P05 Add metrics/instrumentation — track [LATENCY / ERROR RATE / THROUGHPUT] for this function.

Understand and learn:

I01 Explain this code line by line in plain English — assume I am a junior developer.

I02 What are the edge cases this implementation does NOT handle?

I03 What would a senior engineer critique about this code?

I04 What design pattern is this using and why is it appropriate here?

I05 How would you test this in production to verify it works correctly at scale?

Extend and scale:

E01 Extend this to support [NEW FEATURE]. Minimize changes to existing logic.

E02 Refactor this to be more easily testable — reduce coupling to [DEPENDENCY].

E03 Adapt this code to work in a [SERVERLESS / CONTAINERIZED / MULTI-REGION] environment.

E04 Add pagination support to this query/endpoint — use [CURSOR / OFFSET] pagination.

Common Developer Prompt Mistakes

These are the most common reasons developers get wrong, buggy, or outdated code from AI — and exactly how to fix them.

1. No language version specified

MISTAKE: AI generates code for Python 2, outdated JavaScript (var), or an old framework version.
FIX: Always specify: "Python 3.11" / "TypeScript 5.3" / "React 18 + Vite" / "Node.js 20".
RESULT: Code that uses current APIs, syntax, and best practices.

2. No context about the codebase

MISTAKE: AI generates code that conflicts with your existing architecture, patterns, or dependencies.
FIX: Add: "This is part of a [FRAMEWORK] app using [ORM]. It fits into [LAYER / MODULE]."
RESULT: Code that actually integrates with what you already have.

3. Not pasting the actual code

MISTAKE: Describing code instead of pasting it — AI hallucinates what your code looks like.
FIX: Always paste the actual code for debugging, review, refactoring, and test generation.
RESULT: AI works with your real code, not an imagined version of it.

4. Not specifying constraints

MISTAKE: AI uses external libraries you cannot install, or writes code that violates your architecture.
FIX: Add: "No external libraries beyond [LIST]" / "Under [X] lines" / "Must be async-only".
RESULT: Code that fits your project constraints without needing major edits.

5. Trusting the output without reviewing

MISTAKE: Running AI-generated code in production without reading or testing it.
FIX: Always review for: logical errors, security holes, missing error handling, edge cases.
RULE: AI code is a first draft by a smart intern. You are the senior engineer who signs off.

6. Asking for too much at once

MISTAKE: "Build me a full authentication system with OAuth, JWT, refresh tokens, and admin roles."
FIX: Break it into steps: first the data model, then the JWT middleware, then the OAuth flow.
RESULT: Each piece is solid before you build on top of it.

Developer Prompt Quick Cheat Sheet

One page. Every essential element. Keep this open while you code.

The 6-Element Dev Prompt Formula

LANG/VER	TASK	CONTEXT	CONSTRAINTS	FORMAT	EXPLAIN
----------	------	---------	-------------	--------	---------

Essential Context Lines to Always Include

C1 Language: [LANGUAGE + VERSION] Framework: [FRAMEWORK + VERSION]

C2 This code is part of: [WHAT THE LARGER SYSTEM DOES]

C3 It must integrate with: [DATABASE / API / SERVICE]

C4 Constraints: [NO LIBRARIES / MAX LINES / PERFORMANCE REQUIREMENT]

C5 After the code: [EXPLAIN / ADD TESTS / WALK ME THROUGH IT]

Power Phrases That Improve Every Dev Prompt

PP1 Act as a senior [LANGUAGE] developer / security engineer / architect.

PP2 Include type hints / annotations / JSDoc for all functions.

PP3 Add error handling for: null inputs, network failures, and [SPECIFIC EDGE CASE].

PP4 Do NOT use any external libraries beyond [LIST ALLOWED LIBRARIES].

PP5 After the code: explain each function in 1 sentence.

PP6 Add 3 unit tests: happy path, empty input, error case.

PP7 Flag any security considerations I should be aware of.

PP8 Note the time and space complexity of your solution.

Instant Refinement Prompts

R1 Simplify this — it is more complex than it needs to be.

R2 Add error handling — what happens when [X] fails?

R3 This will break with [NULL / EMPTY ARRAY / CONCURRENT REQUESTS] — fix it.

R4 Rewrite this function to be under [X] lines without losing functionality.

R5 There is a security issue here — find and fix it.

Developer Master Quick Reference

Bookmark this page. Your complete developer AI prompting toolkit on one page.

MASTER DEV PROMPT TEMPLATE

```
// Act as a senior [LANGUAGE + VERSION] developer / [SPECIALIST ROLE].
// Task: [WRITE / DEBUG / REVIEW / REFACTOR / EXPLAIN / TEST] [WHAT].
// Context: This is part of [PROJECT TYPE] using [FRAMEWORK + DEPENDENCIES].
// It must integrate with: [DATABASE / API / OTHER SERVICES].
// Constraints:
//   - No external libraries beyond [LIST ALLOWED]
//   - Must handle: [NULL / ERROR / CONCURRENT] cases
//   - Target performance: [BENCHMARK if relevant]
// Output format:
//   - [FUNCTION / CLASS / MODULE / SNIPPET]
//   - With type hints / annotations / JSDoc
//   - With error handling
//   - With [X] unit tests
// After the code: explain each function in one sentence.
// [PASTE YOUR CODE OR DESCRIBE THE TASK]
```

25 Essential Developer Prompt Starters

- 01 Write a [LANGUAGE] function that [DOES X]. Include type hints, error handling, and a docstring.
- 02 Debug this [LANGUAGE] error: [PASTE ERROR]. Code: [PASTE CODE]. Fix and explain the root cause.
- 03 Review this code for: correctness, security, performance, and readability. Score each 1-10.
- 04 Refactor this code for readability without changing behaviour. Show before and after.
- 05 Write [X] unit tests for this function using [FRAMEWORK]. Cover: happy path, edge cases, errors.
- 06 Write a [FRAMEWORK] REST endpoint for [ROUTE]. Include validation, auth, and error responses.
- 07 Optimize this SQL query — it is taking [X] seconds on [ROW COUNT] rows. Suggest indexes.
- 08 Explain this code line by line. Assume I am a junior developer unfamiliar with [PATTERN].
- 09 Write an optimized Dockerfile for a [FRAMEWORK] app. Multi-stage build, non-root user.
- 10 Design a database schema for [SYSTEM]. Include tables, relationships, and indexes.
- 11 Write a GitHub Actions CI workflow for [LANGUAGE] — lint, test, build, deploy to [TARGET].
- 12 Find the security vulnerability in this code and provide the fix: [PASTE CODE]
- 13 Convert this [CALLBACK / PROMISE] code to async/await. Handle all error cases.
- 14 Write a migration using [ALEMBIC / KNEX / FLYWAY] for this schema change: [DESCRIBE CHANGE].

- 15 Write a regex that matches [PATTERN]. Show it in [LANGUAGE] with test cases.
- 16 Implement [DESIGN PATTERN] for [PROBLEM] in [LANGUAGE]. Explain when to use it.
- 17 Do a security audit of this code. Check for: [INJECTION / AUTH / XSS / CSRF / IDOR].
- 18 Write Terraform for [AWS / GCP / AZURE RESOURCE]. Parameterize [VARIABLE LIST].
- 19 Write a commit message for these changes: [DESCRIBE / PASTE DIFF]. Conventional Commits format.
- 20 Explain [TECHNICAL CONCEPT] to a developer who knows [LANGUAGE] but is new to this concept.
- 21 Create a React component for [NAME]. Props: [LIST]. TypeScript, accessible, no inline styles.
- 22 Write Kubernetes manifests for [APP]. Include Deployment, Service, Ingress, and probes.
- 23 Analyze this code for performance issues. What is the Big O? How do I optimize the bottleneck?
- 24 Write a PR description for this change: [DESCRIBE]. Include: what, why, how to test, risks.
- 25 Before you write the code: ask me up to 5 questions to clarify requirements.

AI is your pair programmer. You are still the engineer.

Master AI Prompts. Learn Smarter. Work Faster. — Aicadmia